

Note: This practical has been performed within the Databricks Free Version due to lacking of free credits within Azure.

- 1) Create the catalog and a volume to store the sales dataset and the copied the path for usage within the Notebooks :

The screenshot displays the Databricks Catalog Explorer interface. On the left, a sidebar lists navigation options: Home, Learn, Workspace, Recents, Catalog (selected), Jobs & Pipelines, Compute, Discover (Beta), Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, and SQL Warehouses. The main area is titled 'Catalog Explorer > data > default > data_sales'. Below this, there's a search bar and a list of catalogs under 'My organization'. The 'data_sales' catalog is selected, showing its contents: 'information_schema', 'Delta Shares Received', and 'samples'. The 'data_sales' catalog is expanded, showing a volume '/Volumes/data/default/data_sales'. This volume contains a file named 'retail_sales_data.csv' with a size of 7.62 KB and a last modified time of 6 minutes ago. The interface also includes a 'Description' section with an 'AI generate' button, a 'Filter files and directories at this level' search bar, and an 'About this volume' section showing the owner as 'anujdemo230304@gmail.com'. There are also buttons for 'Share', 'Upload to this volume', 'Refresh', 'Create directory', and 'Add tags'.

Catalog Explorer > data > default > data_sales

data_sales [Share] [Upload to this volume]

Overview Files Details Permissions

Description

[AI generate] [Add]

/Volumes/data/default/data_sales [Refresh] [Create directory]

Filter files and directories at this level

Name	Size	Last modified
retail_sales_data.csv	7.62 KB	6 minutes ago

About this volume

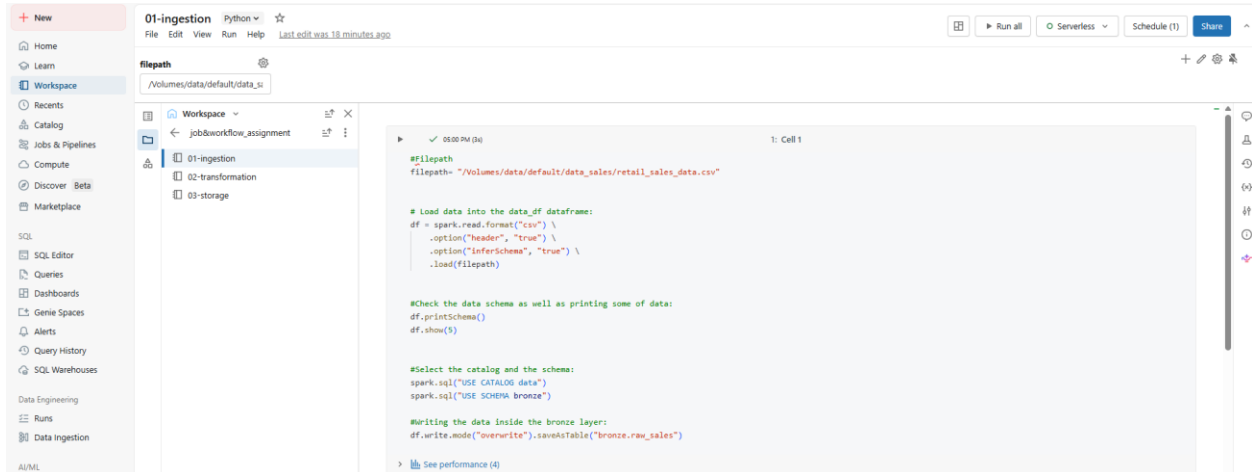
Owner anujdemo230304@gmail.com [edit]

Tags

[Add tags]

2) Create notebooks for the pipeline as mentioned within the assignment:

A) 01-ingestion file for ingestion of data from the volume to data frame:



The screenshot shows a Databricks notebook interface. The left sidebar contains a navigation menu with options like Home, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, and SQL Warehouse. The main area displays the notebook '01-ingestion' with a file path of '/Volumes/data/default/data_sales'. The code in the notebook is as follows:

```
#filepath
filepath= "/Volumes/data/default/data_sales/retail_sales_data.csv"

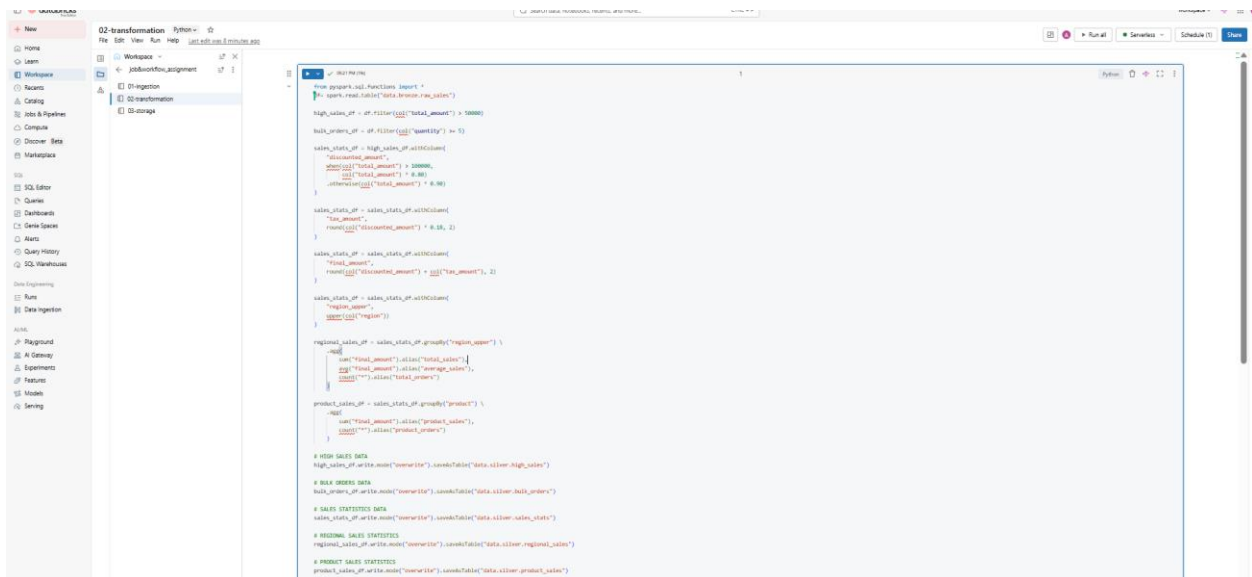
# Load data into the data_df dataframe:
df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(filepath)

#Check the data schema as well as printing some of data:
df.printSchema()
df.show(5)

#Select the catalog and the schema:
spark.sql("USE CATALOG data")
spark.sql("USE SCHEMA bronze")

#Writing the data inside the bronze layer:
df.write.mode("overwrite").saveAsTable("bronze.raw_sales")
```

B) 02-transformation file for transformation of data from the “bronze.raw_sales” to silver layer:



The screenshot shows a Databricks notebook interface. The left sidebar is the same as in the previous image. The main area displays the notebook '02-transformation' with a file path of '/Volumes/data/default/data_sales'. The code in the notebook is as follows:

```
from pyspark.sql.functions import *
spark.read.format("csv").load("bronze.raw_sales")

high_sales_df = df.filter(col("total_amount") > 50000)

sales_stats_df = high_sales_df.withColumn(
    "discounted_amount",
    (col("total_amount") + 100000) / (col("total_amount") + 8.88)
)

sales_stats_df = sales_stats_df.withColumn(
    "tax_amount",
    round(col("discounted_amount") * 8.88, 2)
)

sales_stats_df = sales_stats_df.withColumn(
    "total_amount",
    round(col("discounted_amount") + col("tax_amount"), 2)
)

sales_stats_df = sales_stats_df.withColumn(
    "region_name",
    round(col("region"))
)

regional_sales_df = sales_stats_df.groupBy("region_name") \
    .agg(
        sum("total_amount").alias("total_sales"),
        sum("total_amount").alias("avg_age_sales"),
        sum("total_amount").alias("total_gross")
    )

product_sales_df = sales_stats_df.groupBy("product") \
    .agg(
        sum("total_amount").alias("product_sales"),
        sum("total_amount").alias("product_gross")
    )

# HIGH SALES DATA
high_sales_df.write.mode("overwrite").saveAsTable("data.silver_high_sales")

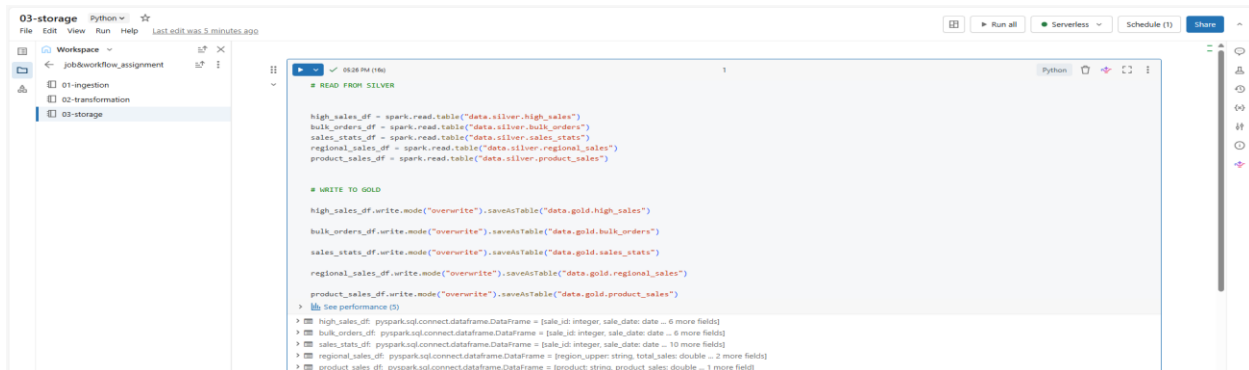
# BULK INDEXES DATA
bulk_indexes_df.write.mode("overwrite").saveAsTable("data.silver_bulk_indexes")

# SALES STATISTICS DATA
sales_stats_df.write.mode("overwrite").saveAsTable("data.silver_sales_stats")

# REGIONAL SALES STATISTICS
regional_sales_df.write.mode("overwrite").saveAsTable("data.silver_regional_sales")

# PRODUCT SALES STATISTICS
product_sales_df.write.mode("overwrite").saveAsTable("data.silver_product_sales")
```

C) 03-storage file for storage of data from the “silver layer” to the “gold layer”:



```
03-storage Python
File Edit View Run Help Last edit was 5 minutes ago

Workspace
job_workflow_assignment
01-ingestion
02-transformation
03-storage

# READ FROM SILVER

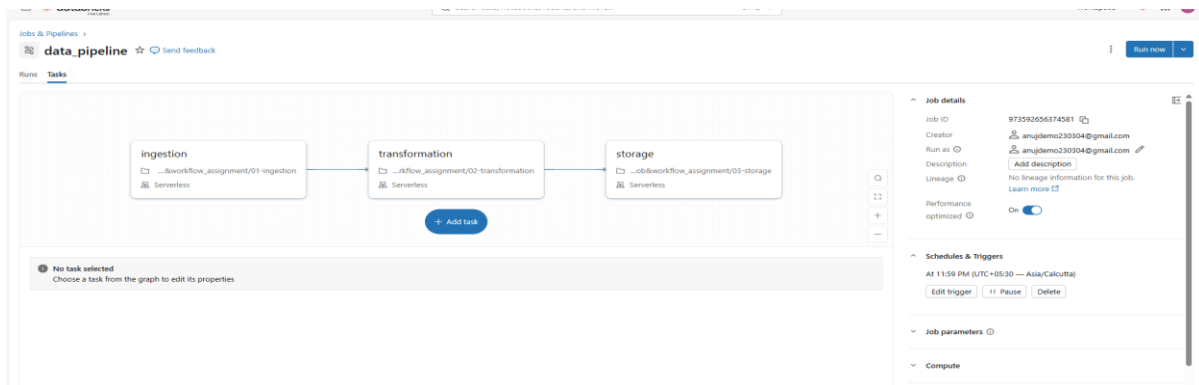
high_sales_df = spark.read.table("data.silver.high_sales")
bulk_orders_df = spark.read.table("data.silver.bulk_orders")
sales_stats_df = spark.read.table("data.silver.sales_stats")
regional_sales_df = spark.read.table("data.silver.regional_sales")
product_sales_df = spark.read.table("data.silver.product_sales")

# WRITE TO GOLD

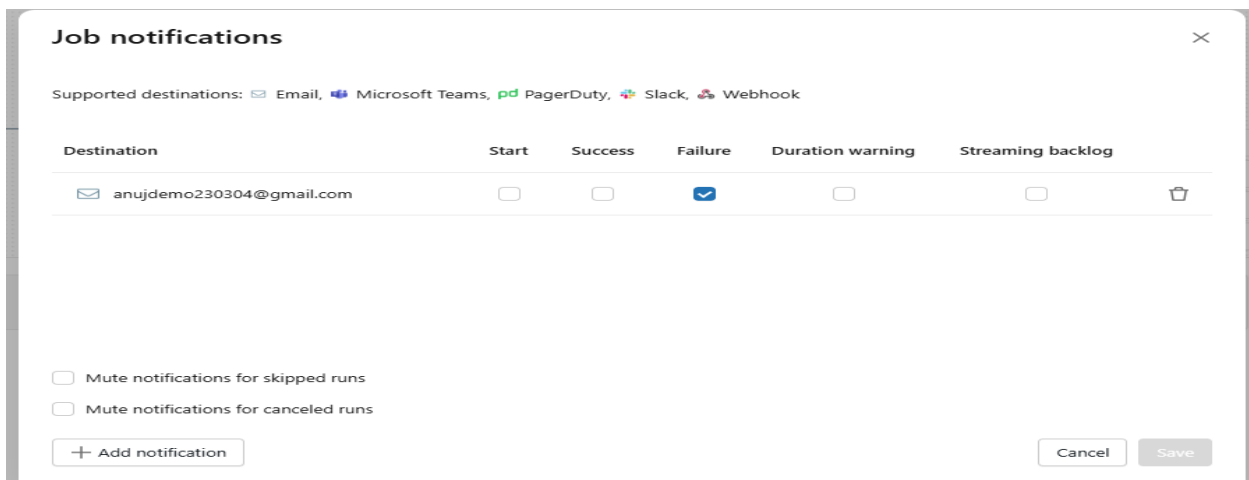
high_sales_df.write.mode("overwrite").saveAsTable("data.gold.high_sales")
bulk_orders_df.write.mode("overwrite").saveAsTable("data.gold.bulk_orders")
sales_stats_df.write.mode("overwrite").saveAsTable("data.gold.sales_stats")
regional_sales_df.write.mode("overwrite").saveAsTable("data.gold.regional_sales")
product_sales_df.write.mode("overwrite").saveAsTable("data.gold.product_sales")

See performance (3)
high_sales_df: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, sale_date: date ... 6 more fields]
bulk_orders_df: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, sale_date: date ... 6 more fields]
sales_stats_df: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, sale_date: date ... 10 more fields]
regional_sales_df: pyspark.sql.connect.dataframe.DataFrame = [region_upper: string, total_sales: double ... 2 more fields]
product_sales_df: pyspark.sql.connect.dataframe.DataFrame = [product: string, product_sales: double ... 1 more fields]
```

3) Created the Job along with the scheduled trigger:



Also added failure-based notifications:



The screenshot shows the "Job notifications" configuration dialog. It lists supported destinations: Email, Microsoft Teams, PagerDuty, Slack, and Webhook. A table shows the configuration for a notification destination: anujdemo230304@gmail.com. The table has columns for Destination, Start, Success, Failure, Duration warning, and Streaming backlog. The Failure column is checked. Below the table, there are checkboxes for "Mute notifications for skipped runs" and "Mute notifications for canceled runs". At the bottom, there is an "Add notification" button and "Cancel" and "Save" buttons.

Destination	Start	Success	Failure	Duration warning	Streaming backlog
anujdemo230304@gmail.com	☐	☐	☒	☐	☐

4) Architecture:

BRONZE LAYER

Raw ingestion layer

(data.bronze.raw_sales)



SILVER LAYER

Clean + structured data

(

- data.silver.high_sales
- data.silver.bulk_orders
- data.silver.sales_stats
- data.silver.regional_sales
- data.silver.product_sales

)



GOLD LAYER

Business-ready analytics

(

- data.gold.high_sales
- data.gold.bulk_orders
- data.gold.sales_stats
- data.gold.regional_sales
- data.gold.product_sales

)

5) Logs from the pipeline (manual trigger based):

The screenshot displays the 'data_pipeline run' interface in Databricks. The main area shows a graph view of the pipeline with three stages: 'ingestion' (Succeeded - 18s), 'transformation' (Succeeded - 16s), and 'storage' (Succeeded - 14s). Each stage is represented by a box with a green checkmark and a list of tasks. The 'ingestion' stage has one task: '...dbworkflow_assignment/01-ingestion'. The 'transformation' stage has one task: '...kflow_assignment/02-transformation'. The 'storage' stage has one task: '...dbworkflow_assignment/03-storage'. All tasks are marked as 'Serverless'. On the right side, the 'Job run details' panel shows the job ID '973592656374581', job run ID '274527780661877', and a status of 'Succeeded'. It also shows the start and end times, duration, and lineage information. Below the job run details, the 'Compute' section shows the cluster 'Cluster terminated' and 'Serverless'.

Catalog and Schema :

The screenshot shows the 'Catalog' view in Databricks. The top bar indicates the current catalog is 'Serverless Starter Warehouse' and the current schema is 'Serverless'. Below the top bar, there is a search bar and a list of data sources. The list is organized into a hierarchy: 'My organization' (workspace, system, data), 'bronze' (raw_sales), 'default', 'gold' (bulk_orders, high_sales, product_sales, regional_sales, sales_stats), 'information_schema', 'silver' (bulk_orders, high_sales, product_sales, regional_sales, sales_stats), and 'Delta Shares Received' (samples). The 'All' tab is selected, and the 'For you' section is visible.